Лабораторная работа №4

Взаимодействие PHP и MySQL. Механизм сессий.

Цель работы: ознакомиться с основными функциями PHP, применяемыми для работы с MySQL-сервером, изучить и применить на практике механизм сессий.

Основные понятия MySQL

СУБД MySQL - одна из множества баз данных, поддерживаемых в PHP. Система MySQL распространяется бесплатно и обладает достаточной мощностью для решения реальных задач.

SQL - это аббревиатура от слов *Structured Query Language*, что означает структурированный язык запросов. Этот язык является стандартным средством для доступа к различным базам данных.

Система MySQL представляет собой сервер, к которому могут подключаться различные серверные скрипты.

Для администрирования баз данных удобно пользоваться средством, входящим в комплект Web-разработчика: Denwer phpMyAdmin. Здесь можно создать новую базу данных, создать новую таблицу в выбранной базе данных, заполнить таблицу данными, а также добавлять, удалять и редактировать данные.

В MySQL определены три базовых типа данных: числовой, дата и время и строчный. Каждая из этих категорий подразделяется на множество типов. Основные из них:

Тип	**Описание**
INT	Целое число
TINYINT	Маленькое целое число (-127 до 128 или от 0 до 255)
FLOAT	Вещественное число с плавающей точкой
DATE	Дата. Отображается в виде ГГГГ-ММ-ДД
TIME	Время. Отображается в виде ЧЧ:ММ:СС
DATETIME	Дата и время. Отображается в виде ГГГГ-ММ-ДДЧЧ:ММ:СС
YEAR[(2|4)]	Год. Можно определить двух- или четырехциферный формат
CHAR(M)	Строка фиксированной длины M (M<=255)
VARCHAR(M)	Строка произвольной длины до M (M<=255)
TEXT	Длинные текстовые фрагменты (<=65535)
BLOB	Большие двоичные объекты (изображения, звуки)

Каждый столбец после своего типа данных содержит и другие спецификаторы:

Тип	Описание
NOT NULL	Все строки таблицы должны иметь значение в этом атрибуте. Если не указано, поле может быть пустым (NULL)
AUTO_INCREMENT	Специальная возможность MySQL, которую можно задействовать в числовых столбцах. Если при вставке строк в таблицу оставлять такое поле пустым, MySQL автоматически генерирует уникальное значение идентификатора. Это значение будет на единицу больше максимального значения, уже существующего в столбце. В каждой таблице может быть не больше одного такого поля. Столбцы с AUTO_INCREMENT должны быть проиндексированными
PRIMARY KEY	Столбец является первичным ключом для таблицы. Данные в этом столбце должны быть уникальными. MySQL автоматически индексирует этот столбец
UNSIGNED	После целочисленного типа означает, что его значение может быть либо положительным, либо нулевым

Создание новой базы данных MySQL

```
 CREATE DATABASE IF NOT EXISTS base
       DEFAULT CHARACTER SET utf8 COLLATE utf8_general_ci
```

Создание новой таблицы.

Например, таблица `books` для книжного магазина будет содержать пять полей: ISBN, автор, название, цена и количество экземпляров:

```
CREATE TABLE books (ISBN VARCHAR(13) NOT NULL,
                    PRIMARY KEY (ISBN),
                    author VARCHAR(30),
                    title VARCHAR(60),
                    price FLOAT(4,2),
                    quantity TINYINT UNSIGNED);
```

Чтобы избежать сообщения об ошибке, если таблица уже есть необходимо изменить первую строчку, добавив фразу "IF NOT EXISTS":

```
CREATE TABLE IF NOT EXISTS books ...
```

Добавление данных

```
INSERT INTO books (ISBN, author, title, price, quantity )
           VALUES ('5-8459-0184-7', 'Зандстра Мэт', 'Освой самостоятельно PHP4
за 24 часа', '129', '5');
```

Извлечение (выборка) данных

Оператор `SELECT` извлекает данные из таблицы, выбирая строки, которые отвечают заданному критерию поиска. Оператор `SELECT` сопровождает немалое количество опций и вариантов использования.

Символ `*` означает, что необходимы все поля. Например:

```
SELECT * FROM books;
```

Для получения доступа только к некоторому полю следует указать его имя в инструкции `SELECT`. Например:

```
SELECT author, title, price FROM books;
```

Чтобы получить доступ к подмножеству строк в таблице, следует указать критерий выбора, который устанавливает конструкция `WHERE`. Например, чтобы выбрать имеющиеся в наличии недорогие книги о PHP, надо составить запрос:

```
SELECT * FROM books WHERE price < 200 AND title LIKE '%PHP%' AND quantity != 0;
```

Символ "%" соответствует любому количеству символов, даже нулевых.

Символ "_" Соответствует ровно одному символу.

Для того, чтобы строки, извлеченные по запросу, перечислялись в определенном порядке, используется конструкция `ORDER BY`. Например:

```
SELECT * FROM books ORDER BY price;
```

По умолчанию порядок сортировки идет по возрастанию. Изменить порядок сортировки на обратный можно с помощью ключевого слова `DESC`:

```
SELECT * FROM books ORDER BY price DESC;
```

Сортировать можно и по нескольким столбцам. Вместо названий столбцов можно использовать их порядковые номера:

```
SELECT * FROM books ORDER BY 4, 2, 3;
```

Для группировки данных используется оператор `GROUP BY` с указанием списка полей. Ограничить количество выбираемых значений можно при помощи оператора `LIMIT` с указанием номера начальной записи и количества выбираемых строк.

```
SELECT author, COUNT(*) FROM books GROUP BY author LIMIT 0, 20;
```

Данный запрос выводит количество книг первых двадцати авторов.

Обновление данных

Например, цену всех книг повысили на 10%:

```
UPDATE books SET price = price * 1.1;
```

Конструкция `WHERE` ограничит работу `UPDATE` определенным строками. Например:

```
UPDATE books SET price = price * 1.05 WHERE price <= 250;
```

Удаление данных

Ненужные строки указываются при помощи конструкции `WHERE`. Например, какие-то книги проданы:

```
DELETE FROM books WHERE quantity = 0;
```

Связь PHP с базой данных MySQL

Создав в phpMyAdmin базу данных, можно приступить к подключению этой базы данных к внешнему Web-интерфейсу.

Чтобы получить доступ к базе данных из Web, используя PHP, надо сделать следующие основные шаги:

- Подключение к серверу MySQL.
- Выбор базы данных.
- Выполнение запроса к базе данных:
  - **добавление;**
  - **удаление;**
  - **изменение;**
  - **поиск;**
  - **сортировка.**
- Получение результата запроса.
- Отсоединение от базы данных.

Для подключения к серверу базы данных в PHP есть функция `mysql_connect( )`. Ее аргументы: имя компьютера, имя пользователя и пароль. Эти аргументы можно опустить. По умолчанию в Denwer имя компьютера **localhost**, имя пользователя **root**, пароль пустой. Функции `mysql_connect()` возвращает идентификатор подключения типа resource, если все прошло успешно. Например:

```
$link = mysql_connect ("localhost", "root", "");
if ( !$link ) die ("Невозможно подключение к MySQL");
```

После того, как соединение с сервером MySQL установлено, нужно выбрать базу данных. Для этого используется функция `mysql_select_db()`. Ее аргумент: имя базы данных.

Функция возвращает истину, если указанная база данных существует и доступ к ней возможен. Например:

```
$db = "sample";
mysql_select_db ($db) or die ("Невозможно открыть $db");
```

Для добавления, удаления, изменения и выбора данных нужно сконструировать в виде строковой переменной запрос SQL и выполнить его. Для этого в языке PHP существует функция `mysql_query()`. Ее аргумент: строка с запросом. Функция возвращает дескриптор результата запроса типа resource при успешном выполнении запроса на выборку, true при успешном выполнении других запросов, false в случае неудачи.

Для закрытия соединения с MySQL используется оператор `mysql_close()`.

Как вы могли заметить, имена всех функций PHP для работы с СУБД MySQL начинаются с префикса `mysql_`. Начиная с версии PHP 5.5, расширение MySQL считается устаревшим и вместо него рекомендуется использовать новую библиотеку MySQLi. Имена большинства функций и их параметры при этом остались такими же. Изменился лишь префикс с `mysql_` на `mysqli_`.

Пример:

```
<html>
<head>
  <title>Пример</title>
</head>

<body>
<?
/* Скрипт показывает пользователей, зарегистрировавшихся после 1 мая 2010г. */

/* Переменные для соединения с базой данных */
$hostname = "localhost";
$username = "root";
$password = "";
$dbName = "my_database";

/* создать соединение */
mysql_connect($hostname,$username,$password) or die("Не могу соединиться с БД");
/* выбрать базу данных. Если произойдет ошибка - вывести ее */
mysql_select_db($dbName) or die(mysql_error());

/* составить запрос, который выберет всех пользователей */
$query = "SELECT * FROM users WHERE date > '2010-05-01'";
/* Выполнить запрос. Если произойдет ошибка - вывести ее. */
$res = mysql_query($query);

/* Как много нашлось таких */
$number = mysql_num_rows($res);

/* Вывести всех в красивом виде*/
if ($number == 0) {
```

```
  echo "<p>Нет таких пользователей</p>";
} else {
  echo "<p>Найдено пользователей: $number<br /><br />";
//Получать по одной записи из результата в массив $row, пока записи не кончатся
 while ($row=mysql_fetch_array($res)) {
    echo "Логин ".$row['login']." | ";
    echo "Его Email: ".$row['email']. " | ";
    echo "Дата регистрации: ".$row['date'];
    echo "<br /><br />";
  }
  echo "</p>";
}
?>
</body>
</html>
```

Результат запроса представляется в виде таблицы, где столбцами выступают названия полей таблицы БД, а строками выбранные записи. Функции для обработки результата запроса (начинаются на `mysql_fetch_`) за один вызов возвращают одну запись в виде массива или объекта. Поэтому обработка результата запроса всегда происходит в цикле. Как только будет достигнут конец записей, функция вернет false и цикл окончится.

Список MySQL-функций:

mysql_affected_rows - получает количество рядов, задействованных в предыдущей операции MySQL
mysql_character_set_name - возвращает имя набора символов (кодировку)
mysql_close - закрывает MySQL-соединение
mysql_connect - открывает соединение с MySQL-сервером
mysql_drop_db - удаляет БД MySQL
mysql_errno - возвращает числовое значение сообщения об ошибке из предыдущей MySQL-операции
mysql_error - возвращает текст сообщения об ошибке из предыдущей MySQL-операции
mysql_escape_string - мнемонизирует/Escapes строку для использования в mysql_query
mysql_fetch_array - возвращает одну запись результата запроса как ассоциативный массив (ключами будут имена полей в таблице БД), числовой массив или оба
mysql_fetch_assoc - возвращает одну запись результата запроса ассоциативный массив
mysql_fetch_field - получает информацию столбца из результата и возвращает как объект
mysql_fetch_object - возвращает одну запись результата запроса как объект. Поля объекта будут именоваться также как поля в таблице БД.
mysql_fetch_row - возвращает одну запись результата запроса в обычном массиве
mysql_field_flags - получает флаги, ассоциированные со специфицированным полем в результате
mysql_field_name - получает имя специфицированного поля в результате
mysql_field_table - получает имя таблицы, в которой находится специфицированное поле
mysql_field_type - получает тип специфицированного поля в результате
mysql_free_result - освобождает результирующую память
mysql_info - получает информацию о самом последнем запросе
mysql_insert_id - получает id, сгенерированный предыдущей операцией INSERT

mysql_list_tables - список таблиц в БД MySQL
mysql_num_fields - получает количество полей в результате
mysql_num_rows - получает количество рядов в результате
mysql_pconnect - открывает постоянное соединение с MySQL-сервером
mysql_query - отправляет MySQL query
mysql_real_escape_string - мнемонизирует/Escapes специальные символы в строке для использования в SQL-операторе, учитывая текущий charset соединения
mysql_result - получает результирующие данные из одной ячейки результата запроса
mysql_select_db - выбирает БД MySQL
mysql_tablename - получает имя таблицы поля

Механизм сессий

Веб-сервер не поддерживает постоянного соединения с клиентом, и каждый запрос обрабатывается, как новый, **безо всякой связи с предыдущими**. То есть, нельзя ни отследить запросы от одного и того же посетителя, ни сохранить для него переменные между просмотрами отдельных страниц, т.к. сборщик мусора PHP уничтожает все переменные после окончания работы скрипта. Вот для решения этих двух задач и были изобретены сессии.

Собственно, сессии - это механизм, позволяющий однозначно идентифицировать конкретного пользователя в определенном браузере на некотором компьютере. При переходе пользователя с одной страницы на другую, данные сессии сохраняются во временном файле на сервере и извлекаются оттуда запрошенным скриптом.

Области применения: корзина покупок в интернет-магазине, авторизация пользователей, защита интерактивных частей сайта от спама (капча).

Как устроены, и как работают сессии

Для начала надо как-то идентифицировать браузер. Для этого надо выдать ему уникальный идентификатор и попросить передавать его с каждым запросом. Идентификатор - это обычная переменная. По умолчанию ее имя - PHPSESSID.

Задача PHP отправить ее браузеру, чтобы тот вернул ее со следующим запросом. Переменную можно передать только двумя способами: в куках или POST/GET запросом.

PHP использует один из вариантов исходя из настроек в php.ini:

`session.use_cookies` - если равно 1, то PHP передает идентификатор в куках. `session.use_trans_sid` - если равно 1, то PHP передает его, добавляя к URL и формам. Этот вариант на данный момент практически не используется.

Если включена первая настройка, то при старте сессии (при каждом вызове функции `session_start()`) клиенту отправляется кука, включающая имя (PHPSESSID), значение сгенерированного уникального идентификатора, домен, путь, время истечения (по умолчанию 24 минуты от текущего времени). Браузер исправно при каждом следующем запросе эту куку возвращает на сервер и PHP имеет идентификатор сессии. В этом случае отсутствия куки PHP будет все время стартовать новую сессию, и механизм работать не будет.

К идентификатору привязывается созданный на сервере временный файл (имя файла совпадает с идентификатором, а также содержит префикс sess_) с данными. PHP это сделает автоматически. Достаточно просто написать:

```
session_start();
$_SESSION['test']='Hello world!';
```

И PHP запишет в файл, связанный с этой сессией, переменную test.

Чтобы поместить переменную в сессию, достаточно присвоить ее элементу массива $_SESSION. Чтобы получить ее значение - достаточно обратиться к тому же элементу.

Сборкой мусора - удалением устаревших файлов PHP тоже занимается сам. Как и кодированием данных и кучей всяких других нужных вещей. В результате этой заботы работа с сессиями оказывается очень простой. Например:

```
<?
session_start();
if (!isset($_SESSION['counter'])) $_SESSION['counter']=0;
echo "Вы обновили эту страницу ".$_SESSION['counter']++." раз. ";
echo "<br /><a href=".$_SERVER['PHP_SELF']."обновить</a>";
?>
```

Мы проверяем, есть ли у нас в сессии переменная counter, если нет, то создаем ее со значением 0, а дальше выводим ее значение и увеличиваем на единицу. Увеличенное значение запишется в сессию, и при следующем вызове скрипта переменная будет иметь значение 1, и так далее.

Для того, чтобы иметь доступ к переменным сессии на любых страницах сайта, надо написать только одну строчку в самом начале каждого файла, в котором нам нужны сессии: `session_start()` и далее обращаться к элементам массива $_SESSION. **Эта функция посылает браузеру HTTP-заголовок, поэтому её вызов должен происходить до вывода в**

браузер любого текста (даже пробелов или символов переноса строк). Как правило, старт сессии происходит на всех страницах сайта в первой строке шапки (header.php).

Например, проверка авторизации будет выглядеть примерно так:

```
session_start();
if ($_SESSION['authorized']<>1) {
header("Location: /auth.php");
exit;
}
```

Удаление переменных из сессии осуществляется функцией `unset($_SESSION['var']);`

По сути, всю работу с сессиями в PHP делает функция `session_start()`. Она проверяет была ли прислана кука с идентификатором, генерирует новый если его не было и отсылает браузеру, создает новый или открывает существующий файл сессии, делает доступным для записи глобальный массив $_SESSION и записывает в него данные из файла.

Разрушить сессию можно при помощи функции `session_destroy()`. При этом обнуляется кука, удаляются файл и массив $_SESSION.

Область применения.

Очень важно понимать, для чего сессии стоит использовать, а для чего - нет.

Во-первых, что сессии можно применять только тогда, когда они нужны самому пользователю, а не для того, чтобы чинить ему препятствия.

Во-вторых. Важно четко себе представлять тот факт, что сессия - это сеанс работы с сайтом, так как его понимает человек. Пришел, поработал, закрыл браузер - сессия завершилась. Этому есть и техническое объяснение. В силу ограничений, механизм сессий не может определить тот момент, когда пользователь закрыл браузер. Для этого используется таймаут – заранее определенное время, по истечении которого мы считаем, что пользователь ушел с сайта. По умолчанию этот параметр равен 24 минутам. Если вы хотите сохранять пользовательскую информацию на более длительный срок, то используйте куки и, если надо - базу данных на сервере. В частности, именно так работают все популярные системы авторизации:

- По факту идентификации пользователя стартует сессия и признак авторизованности передается в ней.
- Если надо "запомнить" пользователя, то ему ставится более длительная кука, его идентифицирующая.

- При следующем заходе пользователя на сайт, для того, чтобы авторизоваться, он должен либо ввести пароль, либо система сама его опознает по поставленной ранее куке, и стартует сессию. Новую сессию, а не продолжая старую.

В-третьих, не стоит стартовать сессии без разбору, каждому входящему на сайт. Это создаст совершенно лишнюю нагрузку. Не используйте сессии по пустякам – к примеру, в счетчиках.

Пример авторизации пользователя:

login.php

```
<?php session_start(); ?>
<html>
      <head>
            <title>Страница авторизации<title>
      </head>
      <body
      <?php
           if (isset($_SESSION["message"])) {
              echo "<div>" . $_SESSION["message "] . "</div>";
              unset($_SESSION["message"]);
           }
      ?>
      <form action="authorize.php" method="post">
            Вход<br />
            Пользователь <input type="text" name="login" /><br />
            Пароль <input type="password" name="pass" />
            <input type="submit" name="ok" value="  Ok  " />
      </form>
      </body>

</html>
```

authorize.php

```
<?php
// соединяемся с базой
$conn = mysql_connect("localhost","root","");
mysql_select_db("my_db");
// составляем запрос
$query = "SELECT * FROM users WHERE login='". $_POST['login']."' AND pass='".
md5($_POST['pass'])."';";
$q = mysql_query($query);
// найден ли кто-нибудь
$n = mysql_num_rows($q);
if ($n!==0)
{
      // стартуем сессию (можно не в начале файла, т.к. никакого вывода в
браузер не было)
    session_start();
    $value=mysql_fetch_array($q);
```

```
      // записываем логин и емейл в сессию
    $_SESSION['user_login']=$value['login'];
    $_SESSION['email']=$value['email'];
      // редиректим (перенаправляем) на главную страницу сайта
    $_SESSION['message']= 'Вы успешно авторизованы';
    Header("Location: index.php");
} else {
    $_SESSION['message']= 'Неверный логин/пароль';
    Header("Location: login.php"); // если юзер не найден, то снова на страницу
авторизации
}
mysql_close();
?>
```

index.php

```
<?php session_start(); ?>
<html>
      <head>
            <title>Главная<title>
      </head>
      <body>
           <?php
           if (isset($_SESSION["message"])) {
              echo "<div>" . $_SESSION["message "] . "</div>";
              unset($_SESSION["message"]);
           }
           ?>
            <?php if ($_SESSION["login"]) echo "<h1>Привет, " .
$_SESSION["login"] . "</h1>"; ?>
<!-- весь остальной контент -->
      </body>

</html>
```

Если пользователь авторизуется, то в index выведет строку приветствия и соответствующее сообщение.

Список функций для работы с сессиями:

session_decode - декодирует данные сессии из строки
session_destroy - уничтожает все данные, зарегистрированные для сессии
session_encode - шифрует данные текущей сессии как строку
session_get_cookie_params - получает параметры куки сессии
session_id - получает и/или устанавливает текущий session id
session_name - получает и/или устанавливает имя текущей сессии
session_save_path - получает и/или устанавливает путь сохранения текущей сессии
session_set_cookie_params - устанавливает параметры куки сессии
session_start - инициализирует данные сессии
session_unset - освобождает все переменные сессии

Задание к лабораторной работе:

1. Изменить web-сайт, разработанный в лабораторной работе №3, так, чтобы названия и контент всех текстовых страниц хранились в базе данных, и администратор мог её редактировать.
2. Создать через phpMyAdmin новую базу данных и все необходимые таблицы.
3. Перенести все названия и тексты страниц в соответствующую таблицу (таблицы).
4. Реализовать регистрацию и авторизацию пользователей на сайте.
5. Реализовать интерфейс администратора (http://site_name/admin/) для управления всеми зарегистрированными пользователями *и текстовыми страницами сайта* (добавление, редактирование, удаление). Использовать механизм сессий. Реализовать разграничение прав доступа (администратор, обычный пользователь).
6. Дополнить задание, выполненное в лабораторной работе №3, используя СУБД MySQL, с учетом требований, обозначенных подчеркиванием:

   отправка email'а через форму обратной связи с полями «Представьтесь», «Ваш E-mail», «Текст сообщения». Использовать несколько дополнительных заголовков и кодировку UFT-8. Данные полей должны проверяться на правильность.
   Письма могут отправлять только авторизованные пользователи и все их письма сохраняются в БД. Администратор может их просмотреть, удалить или отправить еще раз.

   - *RSS-поток новостей с возможностью просмотра в RSS-агрегаторе или браузере.*
     Новости для потока читать из базы данных, создав предварительно систему новостей, которые могут добавлять администратор и авторизованные пользователи. Редактировать и удалять новости может только администратор.
   - *наложение текстов/изображений на другие изображения (водяные знаки), загружаемые с компьютера пользователя, с отображением ссылки на вновь полученный файл.*
     Работать с изображениями могут только авторизованные пользователи. Все ссылки на загруженные и созданные изображения хранятся в БД, администратор может их просмотреть и/или удалить.
   - *рисование диаграмм по информации из таблиц в базе данных. Вид диаграммы должен выбираться пользователем (график, гистограмма).*
     Добавлять исходные данные для диаграмм могут авторизованные пользователи и администратор. Удалять и редактировать записи может только администратор.
   - *организовать сохранение/чтение/удаление информации из базы данных по определенной предметной области (например база данных аудиотреков, видеофильмов, компьютерных комплектующих и т.д.). Список таблиц и хранимых атрибутов (полей) определить самостоятельно.*
     Администратор может управлять всеми записями. Пользователи могут добавлять и редактировать только свои записи.
   - *из передаваемого скрипту статичного html-файла или URL web-страницы со списком новостей используя регулярные выражения в формате PCRE выбрать все новости, сохранить их базе данных и вывести на свою web-страницу игнорируя ненужную служебную информацию, такую как строка меню, шапка заголовка, подвал, т.е. все что не является новостями.*

Записи в БД (список новостей) может отредактировать или удалить администратор сайта. Задавать html-файл или URL web-страницы могут авторизованные пользователи и администратор.

- *ведение статистики по посетителям сайта (ip-адрес, название и версия браузера, количество посещений, заходов на конкретную страницу, дата/время входа). Данные сохранять в таблицу(ы) БД и представить на PHP-странице в удобном виде с возможностью группировки данных и выводом итоговых значений. Предусмотреть отображение в различных разрезах (по времени, по датам, по браузеру, по ip-адресу) в процентных соотношениях.*
  Просматривать статистику могут только авторизованные пользователи. Администратор может управлять всеми записями и имеет возможность фильтрации по полям при просмотре списка посещений.

7. Страницы должны быть работоспособны в любом браузере.

Каждый отчет должен содержать:

1. Заголовок лабораторной работы (название и цель работы).
2. Фамилия, инициалы и группа студента.
3. Задание к лабораторной работе.
4. Краткие теоретические сведения.
5. Описание алгоритмов, функций, примененных решений.
6. Дамп базы данных.
7. Экранные формы разработанных страниц.
8. Основные тексты скриптов.
9. Выводы по сделанной работе.